PES-VCS

PHASE 1:

Screenshot 1A

```
nehaa@nehaa-VirtualBox:~/PES1UG24CS297-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aaf
c12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5
e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
nehaa@nehaa-VirtualBox:~/PES1UG24CS297-pes-vcs$
```

Screenshot 1B

```
nehaa@nehaa-VirtualBox:~/PES1UG24CS297-pes-vcs$ find .pes/objects -type f
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
```

PHASE 2:

Screenshot 2A

```
nehaa@nehaa-VirtualBox:~/PES1UG24CS297-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

Screenshot 2B

```
nehaa@nehaa-VirtualBox:~/PES1UG24CS297-pes-vcs$ find .pes/objects -type f
.pes/objects/5c/7b643ef05407f7f2ccda5aab56d7c140ad232c662bbc3eed5957c1e75c0079
.pes/objects/6e/f19b41225c5369f1c104d45d8d85efa9b057b53b14b4b9b939dd74decc5321
.pes/objects/fe/76325aa5521b207ebe01e12fd8e9e3abf030cacd5398e3744a3a56a81ad1bd
.pes/objects/cf/2eb8d087e7ce0b41f131ef8e01401bcbf06288f31a0152f7b9cd4be89d80b5
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
nehaa@nehaa-VirtualBox:~/PES1UG24CS297-pes-vcs$ xxd .pes/objects/5c/7b643ef05407f7f2ccda5aab56d7c140ad232c662bbc3eed5957c1e75c0079 | head -20
00000000: 636f 6d6d 6974 2032 3435 0074 7265 6520  commit 245.tree
00000010: 3665 6631 3962 3431 3232 3563 3533 3639  6ef19b41225c5369
00000020: 6631 6331 3034 6434 3564 3864 3835 6566  f1c104d45d8d85ef
00000030: 6139 6230 3537 6235 3362 3134 6234 6239  a9b057b53b14b4b9
00000040: 6239 3339 6464 3734 6465 6363 3533 3231  b939dd74decc5321
00000050: 0a70 6172 656e 7420 6366 3265 6238 6430  .parent cf2eb8d0
00000060: 3837 6537 6365 3062 3431 6631 3331 6566  87e7ce0b41f131ef
00000070: 3865 3031 3430 3162 6362 6630 3632 3838  8e01401bcbf06288
00000080: 6633 3161 3031 3532 6637 6239 6364 3462  f31a0152f7b9cd4b
00000090: 6538 3964 3830 6235 0a61 7574 686f 7220  e89d80b5.author
000000a0: 5045 5320 5573 6572 203c 7065 7340 6c6f  PES User <pes@lo
000000b0: 6361 6c68 6f73 743e 2031 3737 3637 3932  calhost> 1776792
000000c0: 3831 310a 636f 6d6d 6974 7465 7220 5045  811.committer PE
000000d0: 5320 5573 6572 203c 7065 7340 6c6f 6361  S User <pes@loca
000000e0: 6c68 6f73 743e 2031 3737 3637 3932 3831  lhost> 177679281
000000f0: 310a 0a73 6563 6f6e 6420 636f 6d6d 6974  1..second commit
nehaa@nehaa-VirtualBox:~/PES1UG24CS297-pes-vcs$
```

PHASE 3:

Screenshot 3A

```
nehaa@nehaa-VirtualBox:~/PES1UG24CS297-pes-vcs$ ./pes init
./pes add file1.txt file2.txt
./pes status
Initialized empty PES repository in .pes/
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  README.md
  untracked:  commit.c
  untracked:  commit.h
  untracked:  object.c
  untracked:  Makefile
  untracked:  test_tree.c
  untracked:  test_sequence.sh
  untracked:  test_objects.c
  untracked:  pes.h
  untracked:  tree.c
  untracked:  index.h
  untracked:  tree.h
  untracked:  pes.c
  untracked:  index.c
  untracked:  .gitignore
```

Screenshot 3B

```
nehaa@nehaa-VirtualBox:~/PES1UG24CS297-pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776792463 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776792463 6 file2.txt
nehaa@nehaa-VirtualBox:~/PES1UG24CS297-pes-vcs$
```

PHASE 4:

Screenshot 4A

```
nehaa@nehaa-VirtualBox:~/PES1UG24CS297-pes-vcs$ ./pes log
commit 5c7b643ef05407f7f2ccda5aab56d7c140ad232c662bbc3eed5957c1e75c0079
Author: PES User <pes@localhost>
Date:   1776792811

    second commit

commit cf2eb8d087e7ce0b41f131ef8e01401bcbf06288f31a0152f7b9cd4be89d80b5
Author: PES User <pes@localhost>
Date:   1776792810

    first commit
```

Screenshot 4B

```
nehaa@nehaa-VirtualBox:~/PES1UG24CS297-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
.pes/objects/5c/7b643ef05407f7f2ccda5aab56d7c140ad232c662bbc3eed5957c1e75c0079
.pes/objects/6e/f19b41225c5369f1c104d45d8d85efa9b057b53b14b4b9b939dd74decc5321
.pes/objects/cf/2eb8d087e7ce0b41f131ef8e01401bcbf06288f31a0152f7b9cd4be89d80b5
.pes/objects/fe/76325aa5521b207ebe01e12fd8e9e3abf030cacd5398e3744a3a56a81ad1bd
.pes/refs/heads/main
nehaa@nehaa-VirtualBox:~/PES1UG24CS297-pes-vcs$
```

Screenshot 4C

```
nehaa@nehaa-VirtualBox:~/PES1UG24CS297-pes-vcs$ cat .pes/refs/heads/main
cat .pes/HEAD
5c7b643ef05407f7f2ccda5aab56d7c140ad232c662bbc3eed5957c1e75c0079
ref: refs/heads/main
nehaa@nehaa-VirtualBox:~/PES1UG24CS297-pes-vcs$
```

PHASE 5 & 6:

Q5.1: A branch in Git is just a file in .git/refs/heads/ containing a commit hash. Creating a branch is creating a file. Given this, how would you implement pes checkout <branch> — what files need to change in .pes/, and what must happen to the working directory? What makes this operation complex?

A branch in Git is essentially a file stored inside .pes/refs/heads/ that contains the hash of the latest commit. To implement pes checkout <branch>, the system needs to update the .pes/HEAD file so that it points to the selected branch. Then it reads the commit hash from the corresponding branch file and loads the commit object. From that commit, it retrieves the tree structure and reconstructs the working directory by writing all files from the stored blobs. This process is complex because it must safely overwrite existing files in the working directory while ensuring no important uncommitted changes are lost. Handling conflicts and maintaining consistency makes checkout a non-trivial operation.

Q5.2: When switching branches, the working directory must be updated to match the target branch's tree. If the user has uncommitted changes to a tracked file, and that file differs between branches, checkout must refuse. Describe how you would detect this "dirty working directory" conflict using only the index and the object store.

When switching branches, it is important to ensure that the working directory does not contain uncommitted changes that could be overwritten. This can be detected by comparing the current working directory with the index. Using system calls like `stat()`, we can check file metadata such as modification time and size. If these values differ from what is stored in the index, the file is considered modified. If a file exists in the index but not in the working directory, it is considered deleted. If such differences exist and the target branch contains conflicting versions, the checkout operation should be stopped to prevent data loss. Thus, a “dirty” working directory is identified by mismatches between the index and actual files.

Q5.3: "Detached HEAD" means HEAD contains a commit hash directly instead of a branch reference. What happens if you make commits in this state? How could a user recover those commits?

A detached HEAD state occurs when the `.pes/HEAD` file directly stores a commit hash instead of referencing a branch. In this state, any new commits are created normally, but they are not attached to any branch. As a result, these commits can become unreachable once the user switches to another branch. However, they are not immediately lost and can still be recovered. To recover such commits, a user can create a new branch pointing to the current commit or manually update a reference file. This ensures that the commits remain accessible in the repository history.

Q6.1: Over time, the object store accumulates unreachable objects — blobs, trees, or commits that no branch points to (directly or transitively). Describe an algorithm to find and delete these objects. What data structure would you use to track "reachable" hashes efficiently? For a repository with 100,000 commits and 50 branches, estimate how many objects you'd need to visit.

Over time, the object store may contain unused objects that are no longer referenced by any branch. To clean these up, we can perform garbage collection by first identifying all reachable objects. Starting from all branch heads, we traverse through commits, their parent commits, associated trees, and blobs. Every visited object is marked as reachable using a data structure like a hash set. After traversal, any object not in this reachable set can be safely deleted. In a repository with around 100,000 commits, this process would involve visiting all commits along with their corresponding tree and blob objects, making it a large but manageable traversal.

Q6.2: Why is it dangerous to run garbage collection concurrently with a commit operation? Describe a race condition where GC could delete an object that a concurrent commit is about to reference. How does Git's real GC avoid this?

Running garbage collection at the same time as a commit operation can lead to serious issues. For example, if a new commit is being created and its objects are written but not yet fully referenced, the garbage collector might mistakenly consider them unreachable and delete them. This results in data corruption. To prevent this, Git uses techniques such as writing objects first, then updating references atomically, and using lock files during operations. These mechanisms ensure that garbage collection does not interfere with ongoing commit processes and maintains repository integrity.